

# Barrier Certificate Synthesis via Interpolation and Difference-of-Convex Programming <sup>★</sup>

Dedicated to Martin Fränzle on the occasion of his 60th birthday.

Guanhua Lin<sup>1</sup> , Hao Wu<sup>1</sup> , and Naijun Zhan<sup>1,2</sup>  

<sup>1</sup> SKLCS, Institute of Software, University of Chinese Academy of Sciences, China

<sup>2</sup> School of Computer Science, Peking University, Beijing, China  
{lingh, wuhao, znj}@ios.ac.cn

**Abstract.** Barrier certificates play a crucial role in the verification of cyber-physical systems (CPS), particularly in safety-critical scenarios. The underlying principle involves encoding the conditions for a polynomial’s zero sub-level set to be a differential invariant, enabling the application of powerful mathematical optimization techniques. A primary challenge in this approach is that, in general settings, the resulting constraints are non-convex and lack efficient solvers. To address this issue, Wang et al. [48,49] proposed a difference-of-convex (DC) algorithm to approximate a non-convex program by solving a series of convex programs using semidefinite programming (SDP). While this approach offers a guarantee on convergence to local optimums, it suffers from the initial value problem. In this paper, we try to solve this problem based on the key insight that a user-defined polynomial in barrier certificate conditions should be selected to be an interpolant separating the initial region and the unsafe region. A novel approach is proposed to adjust the position of the interpolant via SDP, and the obtained interpolant is fed into the DC algorithm as the initial value. Experiments show that our strategy can significantly improve the performance of the DC algorithm.

## 1 Introduction

Cyber-physical systems (CPS) have become increasingly pervasive in modern society, representing a seamless integration of physical components and computational devices to model real-world systems. These complex systems, encompassing diverse domains such as autonomous vehicles, smart grids, and medical devices, present unique challenges in ensuring safety and reliability. As CPS become more sophisticated, the need for rigorous verification and validation techniques grows ever more critical, especially in safety critical scenarios.

Currently, the verification of CPS as surveyed in [17] is either by model-checking based on computing reachable sets or by theorem proving based on specification logic and (differential) invariant generation. The reachability problem of CPS is undecidable in general [22], albeit with decidable sub-classes (see,

---

<sup>★</sup> This work is funded by the National Key R&D Program of China under grant No.2022YFA1005101 and NSFC under grant No.62192732.

e.g., [29,3,18]) identified in the literature. Most of work along this research line focus on approximation and abstraction of reachable sets with symbolic computation, numeric computation or their combination over bounded time horizon, please refer to [1] and the references therein. In contrast, theorem proving [36,57] offer the advantage of proving safety over an infinite time horizon as well as scalability because of the compositionality, therefore deductive verification draws increasingly attentions recently.

Deductive verification leverages the notion of *differential invariant* [37], a continuous analogy of inductive invariants employed in program verification. Namely, a differential invariant is a set of states that is invariant under the system's dynamics, ensuring that any trajectory originating within it will never leave. If a differential invariant separating the initial set and the unsafe set is identified, it serves as an over-approximation of the set of reachable states and thereby certifies the system's safety. For polynomial systems, Liu et al. [32] established a necessary and sufficient condition on whether a semi-algebraic set is a differential invariant, and based on which invented a complete algorithm to synthesizing differential invariants with a predefined templates via quantifier elimination [45]. However, this algorithm is impractical due to the doubly exponential complexity of real quantifier elimination [14].

In [39], Prajna and Jadbabaie introduced the concept of *barrier certificates* to encode a sufficient condition for the zero sub-level set of a real-valued function to be a differential invariant, thereby witnessing safety. By restricting the real-valued function to specific parametric forms, the invariant synthesis problem can be reduced to solving constraints for parameters, further reduced to convex programming problems. Along this research line, various definitions of barrier certificates have been introduced to pursue a trade-off between tractability and expressiveness [28,27,56,13,42,48,49].

For polynomial systems and polynomial barrier certificate templates, the most prevalent method for encoding barrier certificate conditions into constraints leverages Sum-of-Squares (SOS) techniques. Based on representation theorems such as Putinar's Positivstellensatz [41], the barrier certificate conditions are translated into constraints involving SOS polynomials. These SOS constraints are subsequently reduced to Linear Matrix Inequalities (LMIs) [39,27,13,42] or Bilinear Matrix Inequalities (BMIs) [39,56,12,48,49], contingent upon the specific barrier certificate definitions. One limitation of these approaches is that the relative completeness can only be guaranteed when the system domain is bounded, due to the so-called *Archimedean assumption* in Putinar's Positivstellensatz. A recent work [53] addresses this problem by showing that a system with an unbounded domain can be projected into an equivalent higher dimensional system with a bounded domain, while the barrier certificate conditions for the new system can still be encoded into SOS constraints.

In [48], Wang et al. showed that the weakest barrier certificate conditions can be encoded in BMIs and proposed a difference-of-convex (DC) algorithm to approximate the non-convex program as a sequence of convex programs, which can be efficiently solved using semidefinite programming (SDP). While this approach

offers a guarantee of convergence to local minimum and a worst-case polynomial time complexity upper-bound, its performance is sensitive to the initial value problem. The initial value problem refers to the challenge of selecting a good initial point for the iterative optimization procedure, which can significantly impact the convergence speed and quality of the solution.

This paper extends the line of research on DC-based barrier certificate synthesis. The main contributions of this paper are twofold: (1) a novel comparison of the two primary encoding methods for barrier certificate conditions, and (2) a novel approach to adjust the position of the interpolant multiplier polynomial  $\lambda(\mathbf{x})$  (defined in (13)) to mitigate the limitations of the original DC algorithm. The first contribution involves an important observation: if the multiplier polynomial  $\lambda(\mathbf{x})$  is chosen to be an interpolant between the initial set and the unsafe set, the resulting barrier certificate conditions can be less conservative. Building upon this observation, the second contribution proposes an approach to adjust the position of the interpolant and combine it with the original DC algorithm to give the new **ltp**-DC algorithm.

*Related Works* Since numerical solvers are prone to numerical errors, optimization-based approaches often employ exact methods, such as Satisfiability Modulo Theories (SMT) solvers (such as [33,8]) or numerical SMT solvers (such as [20,21]), to verify the correctness of the obtained results.

Inspired by the expressive power of neural networks, recent works have also explored synthesizing barrier certificates as neural networks [60,35,59,15,58]. These approaches typically leverage a counter-example guided inductive synthesis (CEGAR) framework, comprising a learner and a verifier. The learner synthesizes a candidate barrier certificate, represented by a neural network, from sampled points. The verifier, typically an SMT solver, checks the correctness of the candidate barrier certificate and, if invalid, provides counter-example sample points. While these approaches are less susceptible to numerical errors inherent in optimization-based methods and can handle non-polynomial systems, they face challenges in fine-tuning network parameters and verifying large scale neural barrier certificates using SMT solvers.

The notion of barrier certificates can be readily extended to hybrid systems [39], which encompass both continuous-time dynamics and discrete-time transitions. Furthermore, analogous notions of barrier certificates have been developed for systems with additional complexities, such as control inputs [54,2,11], disturbances [50], randomness [40,23,26], and temporal logic specifications [52,10,55].

Another direction is to extend the proof rules underlying barrier certificates, this leads to the definition of  $t$ -barrier certificates [7] and  $k$ -inductive barrier certificates [4,5], which can be seen as replacing induction by  $k$ -induction, and closure barrier certificates [34], which can be seen as replacing state invariants by transition invariants [38].

*Paper Structure.* The rest of the paper proceeds as follows: In Sect. 2, we fix the notation and introduce necessary concepts. In Sect. 3, we compare two types of barrier certificate conditions that lead to LMIs and BMIs. Sect. 4 recalls the DC

algorithm proposed in [48] and discuss its limitations. In Sect. 5, we propose the **ltp-DC** algorithm by improving the DC algorithm with interpolation techniques, where the crucial part is a SDP-based framework to adjust the position of an interpolant. Sect. 6 provides a case study and, finally, Sect. 7 concludes the paper.

## 2 Preliminaries

We first fix basic notations and then introduce necessary concepts used throughout this paper. Let  $\mathbb{N}$  denote the set of natural numbers. Let  $\mathbb{R}$ ,  $\mathbb{R}_{>0}$ , and  $\mathbb{R}_{\geq 0}$  denote the set of real numbers, the set of positive real numbers, and the set of non-negative real numbers, respectively. By convention, we use boldface letters to denote vectors such as  $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$  and vector-valued functions such as  $\mathbf{f} = (f_1, \dots, f_m)$ . The inner product of two vectors  $\mathbf{x}$  and  $\mathbf{y}$  is denoted by  $\langle \mathbf{x}, \mathbf{y} \rangle$ . We denote by  $\mathbb{R}[\mathbf{x}]$  the set of polynomials in variables  $\mathbf{x}$  and denote by  $\mathbb{R}^d[\mathbf{x}]$  the set of polynomials in variables  $\mathbf{x}$  of degree less than or equal to  $d$ . A basic (closed) semialgebraic set (BSA) is of the form  $\{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\}$ , where  $p_i(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$  for any  $1 \leq i \leq m$ . The zero sub-level set of a real-valued function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  refers to the set  $\{\mathbf{x} \in \mathbb{R}^n \mid f(\mathbf{x}) \leq 0\}$ .

### 2.1 Dynamical Systems and Barrier Certificates

*Differential Dynamical Systems.* We consider continuous-time dynamical systems described by ODEs:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}), \quad (1)$$

where  $\mathbf{x} \in \mathbb{R}^n$  is the state vector,  $\dot{\mathbf{x}}$  denotes the temporal derivative  $dx/dt$ , and  $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$  is a polynomial vector field, i.e., each component  $f_i$  of  $\mathbf{f}$  is a polynomial in  $\mathbb{R}[\mathbf{x}]$ . Since a polynomial vector field is locally Lipschitz continuous, system (1) admits a unique *trajectory*, denoted by

$$\xi_{\mathbf{x}_0} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n \quad (2)$$

from any initial state  $\mathbf{x}_0 \in \mathbb{R}^n$ , satisfying that  $\xi_{\mathbf{x}_0}(0) = \mathbf{x}_0$  and  $\dot{\xi}_{\mathbf{x}_0}(t) = \mathbf{f}(\xi_{\mathbf{x}_0}(t))$  for any  $t \in \mathbb{R}_{\geq 0}$ .

*Safety Verification Problems.* Given system (1) with a domain  $\mathcal{D} \subseteq \mathbb{R}^n$ , an initial set  $\mathcal{I} \subset \mathcal{D}$ , and an unsafe set  $\mathcal{U} \subset \mathcal{D}$ , the *safety verification problem* asks whether  $\mathcal{U}$  is reachable from any state in  $\mathcal{I}$  within  $\mathcal{D}$ . Formally, let **Reach** denote the reachable set, i.e.,

$$\text{Reach} \triangleq \{\mathbf{x} \in \mathcal{D} \mid \exists t \in \mathbb{R}_{\geq 0}, \exists \mathbf{x}_0 \in \mathcal{I}. \mathbf{x} = \xi_{\mathbf{x}_0}(t)\}, \quad (3)$$

where we assume that a trajectory will never leave the domain. The system is said to be *safe* if  $\mathcal{U} \cap \text{Reach} = \emptyset$ , and *unsafe* otherwise.

In the rest of this paper, we will assume that sets  $\mathcal{D}$ ,  $\mathcal{I}$ , and  $\mathcal{U}$  are BSAs defined by

$$\begin{aligned}\mathcal{D} &\triangleq \{\mathbf{x} \in \mathbb{R}^n \mid g_i^{\mathcal{D}}(\mathbf{x}) \geq 0, \text{ for } i = 0, \dots, m_x\}, \\ \mathcal{I} &\triangleq \{\mathbf{x} \in \mathcal{D} \mid g_i^{\mathcal{I}}(\mathbf{x}) \geq 0, \text{ for } i = 0, \dots, m_i\}, \\ \mathcal{U} &\triangleq \{\mathbf{x} \in \mathcal{D} \mid g_i^{\mathcal{U}}(\mathbf{x}) \geq 0, \text{ for } i = 0, \dots, m_u\},\end{aligned}\tag{4}$$

where  $g_0^{\mathcal{D}} = g_0^{\mathcal{I}} = g_0^{\mathcal{U}} = 1$  are redundant. For simplicity, we will further assume that all these sets are compact, i.e., closed and bounded. This assumption can be removed by incorporating another algebraic technique called polynomial homogenization [24].

*Invariants.* A *positive/differential invariant*, or simply *invariant*, is a subset  $\Omega \subseteq \mathcal{D}$  such that any trajectory starting from  $\Omega$  stays within it forever, i.e.,

$$\forall \mathbf{x}_0 \in \Omega, \forall t \in \mathbb{R}_{\geq 0}. \xi_{\mathbf{x}_0}(t) \in \Omega,$$

where we assume that a trajectory will never leave  $\mathcal{D}$ . According to its definition, if an invariant  $\Omega \subseteq \mathcal{D}$  is found such that  $\mathcal{I} \subseteq \Omega$  and  $\mathcal{U} \subseteq \mathcal{D} \setminus \Omega$ , one can prove the system's safety without explicitly computing or approximating the reachable set, which is typically intractable for nonlinear systems [22].

*Barrier Certificates.* Barrier certificates, originally proposed in [39], encode the conditions for a zero sub-level set of a real-valued function to become an invariant, where  $B : \mathcal{D} \rightarrow \mathbb{R}$  is a differentiable continuous function.

**Theorem 1 (Barrier Certificates [39, Thm. 1]).** *Given the system (1) with sets  $\mathcal{D}$ ,  $\mathcal{I}$ , and  $\mathcal{U}$ . The system is safe if there exists a barrier certificate, namely a real-valued differentiable continuous function  $B(\mathbf{x})$  over  $\mathcal{D}$  satisfying the following conditions*

$$\forall \mathbf{x} \in \mathcal{I}. B(\mathbf{x}) \leq 0, \tag{5}$$

$$\forall \mathbf{x} \in \mathcal{U}. B(\mathbf{x}) > 0, \tag{6}$$

$$\forall \mathbf{x} \in \mathcal{D}. B(\mathbf{x}) = 0 \implies \mathfrak{L}_{\mathbf{f}}B(\mathbf{x}) < 0, \tag{7}$$

where  $\mathfrak{L}_{\mathbf{f}}B(\mathbf{x}) \triangleq \langle \nabla B(\mathbf{x}), \mathbf{f}(\mathbf{x}) \rangle$  is the Lie derivative of  $B(\mathbf{x})$  over the vector field  $\mathbf{f}$ .

Intuitively, conditions (5) and (6) assert that  $B(\mathbf{x}) = 0$  serves as a boundary that separates  $\mathcal{I}$  and  $\mathcal{U}$ . In addition, condition (7) requires that no trajectory will cross this boundary from inside out. We remark that the condition (7) in [39, Thm. 1] originally wrote  $\mathfrak{L}_{\mathbf{f}}B(\mathbf{x}) \leq 0$  instead of  $\mathfrak{L}_{\mathbf{f}}B(\mathbf{x}) < 0$ . This is in general incorrect and relies on an additional assumption [43]

$$B(\mathbf{x}) = 0 \implies \nabla B(\mathbf{x}_0) \neq 0, \tag{8}$$

which means that the gradient  $\nabla B(\mathbf{x})$  will not vanish on the boundary  $B(\mathbf{x}) = 0$ .

Note that the conditions (5) to (7) are only sufficient conditions for the zero sub-level set  $\Omega \triangleq \{\mathbf{x} \in \mathbb{R}^n \mid B(\mathbf{x}) \leq 0\}$  to be an invariant. To obtain sufficient and necessary conditions, one need to consider higher-order Lie derivatives in the Lie derivative condition (7), referring to [49, Def. 4] for more details. Nevertheless, synthesizing barrier certificates according to the above theorem is already challenging because the translated constraints are non-convex, explained in Sect. 3.

*Parameterization* To synthesize a barrier certificate subject to the conditions in Theorem 1, we employ the idea of *parameterization*. First, we set a parameterized template for the target barrier certificate, denoted  $B(\mathbf{x}; \mathbf{b})$ , where  $\mathbf{b}$  are called template variables. For computational tractability,  $B(\mathbf{x}; \mathbf{b})$  is usually taken to be polynomials while the template variables  $\mathbf{b}$  represent unknown coefficients in the polynomial. For instance, when  $\mathbf{x} = (x_1, x_2)$ , a quadratic template can be of the form  $B(\mathbf{x}; \mathbf{b}) = b_1 + b_2x_1 + b_3x_2 + b_4x_1^2 + b_5x_1x_2 + b_6x_2^2$  with  $\mathbf{b} \in \mathbb{R}^6$ . Moreover, we can assume  $b_i \in [-1, 1]$  for each element  $b_i \in \mathbf{b}$ , as  $\mathbf{b}$  can be scaled by any positive factor without changing the zero sub-level set of  $B(\mathbf{x}; \mathbf{b})$ .

## 2.2 Sum-of-Squares Polynomials and Putinar's Positivstellensatz

*Sum-of-Squares Polynomials.* We say a polynomial  $p(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$  is a SOS polynomial if it can be expressed as  $p(\mathbf{x}) = \sum_{i=1}^m p_i(\mathbf{x})^2$  for some  $m \in \mathbb{N}$  and  $p_i(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$  for  $1 \leq i \leq m$ . We denote by  $\Sigma[\mathbf{x}]$  the set of SOS polynomials in variables  $\mathbf{x}$  and denote by  $\Sigma^d[\mathbf{x}] \triangleq \Sigma[\mathbf{x}] \cap \mathbb{R}^d[\mathbf{x}]$  the subset of  $\Sigma[\mathbf{x}]$  of degree up to  $d$ .

Let  $\mathbf{m}_d(\mathbf{x})$  denote the column vector of all monomials in  $\mathbf{x} \in \mathbb{R}^n$  of degree up to  $d$ . For example, when  $\mathbf{x} = (x_1, x_2)$ ,  $\mathbf{m}_2(\mathbf{x}) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$ . Then, a polynomial  $p(\mathbf{x}) \in \mathbb{R}^{2d}[\mathbf{x}]$  can be expressed as

$$p(\mathbf{x}) = \mathbf{m}_d^T(\mathbf{x}) C_p \mathbf{m}_d(\mathbf{x}),$$

where  $C_p$  is a symmetric matrix referred to as the *Gram matrix* associated to  $p$ . An important result states that a polynomial  $p$  is an SOS polynomial if and only if its Gram matrix is positive semidefinite (PSD), i.e., all its eigenvalues are non-negative, denoted by  $C_p \succeq \mathbf{0}$  [31, Prop. 2.1].

*Putinar's Theorem.* Let  $\mathcal{K}$  be a BSA as follows:

$$\mathcal{K} \triangleq \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\}, \quad (9)$$

where  $p_i(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$  for each  $i$ . A *quadratic module* associated to  $\mathcal{K}$  is a set of polynomials of the form

$$\mathbf{QM}(\mathcal{K}) \triangleq \left\{ \sigma_0 + \sum_{i=1}^m \sigma_i p_i \mid \sigma_i \in \Sigma[\mathbf{x}] \text{ for } 0 \leq i \leq m \right\}.$$

By the nonnegativity of SOS polynomials, it is straightforward to see that any polynomial in  $\mathbf{QM}(\mathcal{K})$  is nonnegative over  $\mathcal{K}$ . What is nontrivial is that, under

a mild condition,  $\mathbf{QM}(\mathcal{K})$  actually includes all polynomials that are strictly positive over  $\mathcal{K}$ , stated in the following theorem.

**Theorem 2 (Putinar’s Positivstellensatz [41,31]).** *Let  $\mathcal{K}$  be a BSA as in (9) and assume the Archimedean condition holds, i.e.,  $N - \|\mathbf{x}\|^2 \in \mathbf{QM}(\mathcal{K})$  for some  $N \in \mathbb{N}$ . If a polynomial  $f \in \mathbb{R}[\mathbf{x}]$  is strictly positive over  $\mathcal{K}$ , then  $f \in \mathbf{QM}(\mathcal{K})$ .*

Theorem 2 is an important result in real algebraic geometry. For our problems, it offers an approach to translate the barrier certificate conditions into more tractable constraints. In its statement, the Archimedean assumption can be intuitively understood as requiring that the BSA  $\mathcal{K}$  is compact.

*Interpolation.* In the following, we demonstrate the application of Theorem 2 through the synthesis of interpolants, which will be a core concept later. In first-order logic, an *interpolant* between a two formulas  $\varphi(\mathbf{x})$  and  $\psi(\mathbf{x})$  with  $\varphi(\mathbf{x}) \models \psi(\mathbf{x})$  is a new formula  $\tau(\mathbf{x})$  such that  $\varphi(\mathbf{x}) \models \tau(\mathbf{x})$  and  $\tau(\mathbf{x}) \models \psi(\mathbf{x})$ <sup>3</sup>. We extend the notation to say that  $\tau(\mathbf{x})$  is an interpolant between two regions  $S_1$  and  $S_2$  in  $\mathbb{R}^n$  if  $\tau(\mathbf{x}) < 0$  over  $S_1$  and  $\tau(\mathbf{x}) > 0$  over  $S_2$ .

Suppose we want to synthesize a polynomial  $\tau(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$  as an interpolant between  $\mathcal{I}$  and  $\mathcal{U}$  defined in (4), i.e.,  $\tau(\mathbf{x}) < 0$  over  $\mathcal{I}$  and  $\tau(\mathbf{x}) > 0$  over  $\mathcal{U}$ . It is straightforward to see that the above conditions are basically barrier certificate conditions without the Lie derivative condition. By applying Theorem 2, [19, Thm. 5] shows that a polynomial interpolant can be obtained by solving the following program:

$$\begin{aligned} \text{find } & \tau(\mathbf{x}; \mathbf{a}) \\ \text{s.t. } & -\tau(\mathbf{x}; \mathbf{a}) = \sum_{i=0}^{m_i} \sigma_i^{\mathcal{I}}(\mathbf{x}) g_i^{\mathcal{I}}(\mathbf{x}) + \epsilon_0 \\ & \tau(\mathbf{x}; \mathbf{a}) = \sum_{i=0}^{m_u} \sigma_i^{\mathcal{U}}(\mathbf{x}) g_i^{\mathcal{U}}(\mathbf{x}) + \epsilon_0, \end{aligned} \tag{10}$$

where the constant  $\epsilon_0 > 0$  is introduced to distinguish between  $> 0$  to  $\geq 0$ .

By extracting Gram matrices, we obtain the following program:

$$\begin{aligned} \text{find. } & \mathbf{a}, \mathbf{s} \\ \text{s.t. } & -C_{\tau}(\mathbf{a}) = \sum_{i=0}^{m_i} C_{\sigma_i^{\mathcal{I}} \cdot g_i^{\mathcal{I}}}(\mathbf{s}) + C_{\epsilon} \\ & C_{\tau}(\mathbf{a}) = \sum_{i=0}^{m_u} C_{\sigma_i^{\mathcal{U}} \cdot g_i^{\mathcal{U}}}(\mathbf{s}) + C_{\epsilon} \\ & C_{\sigma_i^{\mathcal{I}}}, C_{\sigma_i^{\mathcal{U}}} \succeq 0. \end{aligned} \tag{11}$$

<sup>3</sup> In fact, the Craig interpolation theorem states that both  $\varphi$  and  $\psi$  can contain exclusive variables. In this paper, we focus on the simpler case that both these formulas are in variables  $\mathbf{x}$ .

where, for example,  $C_{\sigma_i^T \cdot g_i^T}$  is the Gram matrix of the product of polynomial  $\sigma_i^T \cdot g_i^T$  for  $0 \leq i \leq m_i$ . The constraints in (11) are LMIs and hence the whole program adheres to the standard definition of SDP [9, Sec. 4.6.2].

### 3 Tractable Barrier Certificate Conditions

Two principal approaches exist to transform the conditions in Theorem 1 into a constraint-solving problem, differentiated by their treatment of the Lie derivative condition (7).

One approach involves strengthening condition (7) to the following form

$$\forall \mathbf{x} \in \mathcal{D}. \mathfrak{L}_f B(\mathbf{x}) \leq \omega(B(\mathbf{x})) \quad (12)$$

where  $\omega$  is a univariate locally Lipschitz continuous function with  $\omega(0) \leq 0$ . This condition was proposed in [13] and further elucidated in [42] using the *comparison principle* [47, p. 96]. The geometric explanation of this condition is illustrated in Fig. 1c: ideally, the function  $\omega(y)$  is chosen such that  $\omega(y) \geq 0$  for  $y \in \mathbb{R}$  and  $\omega(y) = 0$  is exclusively attained at  $y = 0$ . For instance,  $\omega(y) = y^2$  is a suitable choice.

Unfortunately, such an ideal  $\omega$  cannot be directly utilized in the SDP-based synthesis of  $B(\mathbf{x})$ . This is due to the fact that, for nonlinear  $\omega$ ,  $\omega(B(\mathbf{x}; \mathbf{b}))$  introduces nonlinear terms in template variables  $\mathbf{b}$ , rendering the constraints not reducible to SDP (even more complex than BMI defined later). To leverage SDP techniques,  $\omega$  must be linear, taking the form  $\omega(B(\mathbf{x})) = \lambda B(\mathbf{x})$  for some constant  $\lambda \in \mathbb{R}$ . In this case, the condition (12) reduces to the so-called *exponential-type* barrier certificate conditions in [28]. While these exponential-type conditions are straightforward to apply, they are inherently conservative, imposing unnecessary stability requirements:

- When  $\lambda > 0$ , as depicted in Fig. 1a, the exponential-type condition requires  $\mathfrak{L}_f B(\mathbf{x}) \leq \omega(B(\mathbf{x})) < 0$  whenever  $B(\mathbf{x}) < 0$ . Using the classic Lyapunov stability argument, this means that any trajectory starting from the region  $B(\mathbf{x}_0) < 0$  will stabilize to the minimizer of  $B(\mathbf{x})$  (if exists).
- When  $\lambda < 0$ , as depicted in Fig. 1b, the exponential-type condition requires  $\mathfrak{L}_f B(\mathbf{x}) \leq \omega(B(\mathbf{x})) < 0$  whenever  $B(\mathbf{x}) > 0$ . Hence, any trajectory starting from the region  $B(\mathbf{x}_0) > 0$  will stabilize to the region  $B(\mathbf{x}) \leq 0$ .

The other approach, adopted in this paper, is to deal with condition (7) directly. Condition (7) essentially stipulates that  $\mathfrak{L}_f B(\mathbf{x})$  is negative over the BSA defined by  $\mathcal{D} \cap \{\mathbf{x} \in \mathbb{R} \mid B(\mathbf{x}) = 0\}$ . By applying Theorem 2 solely to the boundary  $\{\mathbf{x} \in \mathbb{R} \mid B(\mathbf{x}) = 0\}$ , we know that (7) holds if there exists a multiplier polynomial  $\lambda(\mathbf{x})$  such that

$$\forall \mathbf{x} \in \mathcal{D}. \mathfrak{L}_f B(\mathbf{x}) < \lambda(\mathbf{x})B(\mathbf{x}). \quad (13)$$

Similar to the original condition, the strict inequality above can be relaxed to a nonstrict inequality under the assumption (8).

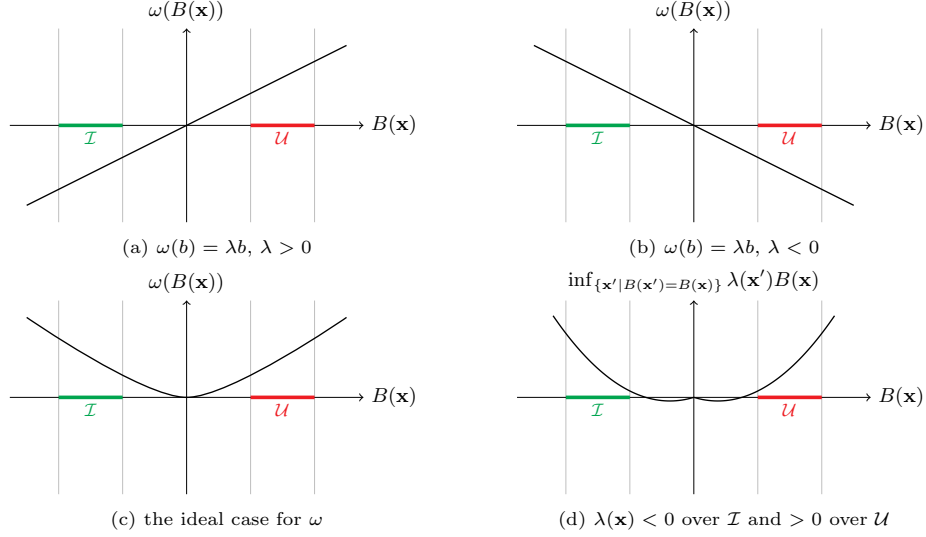


Fig. 1: Different Cases of Barrier Certificate Conditions.

The advantage of (13) lies in its ability to approximate the ideal case of (12) through careful selection of  $\lambda(\mathbf{x})$ . As depicted in Fig. 1d, if  $\lambda(\mathbf{x})$  is an interpolant with  $\lambda(\mathbf{x}) < 0$  over  $\mathcal{I}$  and  $\lambda(\mathbf{x}) > 0$  over  $\mathcal{U}$ , then (13) becomes conservative only in a narrow region between  $\mathcal{I}$  and  $\mathcal{U}$  near the boundary  $B(\mathbf{x}) = 0$ . Applying Theorem 2 again yields the following sufficient condition.

**Theorem 3 (Sufficient Condition).** *Given system (1) with sets  $\mathcal{D}$ ,  $\mathcal{I}$ , and  $\mathcal{U}$ . If there exist SOS polynomials  $\sigma_i^{\mathcal{D}}(\mathbf{x}), \sigma_i^{\mathcal{I}}(\mathbf{x}), \sigma_i^{\mathcal{U}}(\mathbf{x}) \in \Sigma[\mathbf{x}]$ , a multiplier polynomial  $\lambda(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$ , and a positive real constant  $\epsilon_0 > 0$  such that the following conditions hold for  $B(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$ :*

$$-B(\mathbf{x}) = \sum_{i=0}^{m_i} \sigma_i^{\mathcal{I}}(\mathbf{x}) g_i^{\mathcal{I}}(\mathbf{x}) \quad (14)$$

$$B(\mathbf{x}) = \sum_{i=0}^{m_u} \sigma_i^{\mathcal{U}}(\mathbf{x}) g_i^{\mathcal{U}}(\mathbf{x}) + \epsilon_0 \quad (15)$$

$$\lambda(\mathbf{x})B(\mathbf{x}) - \mathfrak{L}_{\mathbf{f}}B(\mathbf{x}) = \sum_{i=0}^{m_d} \sigma_i^{\mathcal{D}}(\mathbf{x}) g_i^{\mathcal{D}}(\mathbf{x}) + \epsilon_0, \quad (16)$$

then the polynomial  $B(\mathbf{x})$  is a barrier certificate.

The above theorem is a special case of [49, Thm. 5], focusing solely on the first-order Lie derivative condition. Higher-order Lie derivatives are not considered due to the significant increase in complexity they introduce to constraint solving. The small constant  $\epsilon_0 > 0$  is introduced to distinguish between  $x > 0$

and  $x \geq 0$ . A necessary condition for Theorem 1 can be similarly derived, even for unbounded domains. For a detailed explanation, we refer the reader to [53].

After fixing the maximum degree of involved polynomials, the conditions in Theorem 3 can be reduced to BMIs. Assume that the unknown SOS polynomials and the multiplier  $\lambda(\mathbf{x})$  are parameterized by template variables  $\mathbf{s}$  and  $\mathbf{a}$ , respectively. By extracting Gram matrices, we obtain the following program:

$$\begin{aligned}
& \text{find.} \quad \mathbf{a}, \mathbf{b}, \mathbf{s} \\
& s.t. \quad -C_B(\mathbf{b}) = \sum_{i=0}^{m_i} C_{\sigma_i^{\mathcal{T}} \cdot g_i^{\mathcal{T}}}(\mathbf{s}) \\
& \quad \quad C_B(\mathbf{b}) = \sum_{i=0}^{m_u} C_{\sigma_i^{\mathcal{U}} \cdot g_i^{\mathcal{U}}}(\mathbf{s}) + C_{\epsilon} \\
& \quad \quad C_{\lambda \cdot B}(\mathbf{a}, \mathbf{b}) - C_{\mathfrak{L}_f B}(\mathbf{b}) = \sum_{i=0}^{m_d} C_{\sigma_i^{\mathcal{P}} \cdot g_i^{\mathcal{P}}}(\mathbf{s}) + C_{\epsilon} \\
& \quad \quad C_{\sigma_i^{\mathcal{T}}}, C_{\sigma_i^{\mathcal{U}}}, C_{\sigma_i^{\mathcal{P}}} \succeq 0.
\end{aligned} \tag{17}$$

The constraints the above program are bilinear due to the existence of cross product terms between  $\mathbf{a}$  and  $\mathbf{b}$  in the entries of  $C_{\lambda \cdot B}(\mathbf{a}, \mathbf{b})$ . To leverage optimization techniques, it is more convenient to formulate the feasible problem as an optimization problem:

$$\begin{aligned}
& \max_{\mathbf{a}, \mathbf{b}, \mathbf{s}, \mu} \quad \mu \\
& s.t. \quad -C_B(\mathbf{b}) - \sum_{i=1}^{m_i} C_{\sigma_i^{\mathcal{T}} \cdot g_i^{\mathcal{T}}}(\mathbf{s}) \succeq \mu I \\
& \quad \quad C_B(\mathbf{b}) - \sum_{i=1}^{m_u} C_{\sigma_i^{\mathcal{U}} \cdot g_i^{\mathcal{U}}}(\mathbf{s}) - C_{\epsilon} \succeq \mu I \\
& \quad \quad C_{\lambda \cdot B}(\mathbf{a}, \mathbf{b}) - C_{\mathfrak{L}_f B}(\mathbf{b}) - \sum_{i=1}^{m_d} C_{\sigma_i^{\mathcal{P}} \cdot g_i^{\mathcal{P}}}(\mathbf{s}) - C_{\epsilon} \succeq \mu I \\
& \quad \quad C_{\sigma_i^{\mathcal{T}}}, C_{\sigma_i^{\mathcal{U}}}, C_{\sigma_i^{\mathcal{P}}} \succeq 0.
\end{aligned} \tag{18}$$

where  $\mu \in \mathbb{R}$  is a newly introduced decision variable as the optimization objective. It is straightforward to see that the optimal value  $\mu^* \geq 0$  of Prog. (18) if and only if Prog. (17) is feasible.

## 4 Difference-of-Convex Algorithm

Difference-of-Convex (DC) programming is a popular technique for addressing non-convex optimization problems [44, 46]. The crux of DC algorithms lies in decomposing a non-convex function into the difference of two convex functions. This decomposition enables the application of powerful convex optimization techniques to approximate solutions of the original non-convex problem.

In this section, we recap the DC algorithm that was proposed in [48] and extended in [49]. The algorithm aims at solving the following optimization problem with BMI constraints:

$$\begin{aligned} \max_{\mathbf{x}, \mathbf{y}} \quad & \mu(\mathbf{x}, \mathbf{y}) \\ \text{s.t.} \quad & \mathcal{B}(\mathbf{x}, \mathbf{y}) \triangleq F + \sum_{i=1}^m x_i H_i + \sum_{j=1}^n y_j G_j + \sum_{i=1}^m \sum_{j=1}^n x_i y_j F_{i,j} \preceq 0. \end{aligned} \quad (19)$$

where  $\mathbf{x} \in \mathbb{R}^m$ ,  $\mathbf{y} \in \mathbb{R}^n$ , the objective function  $\mu : \mathbb{R}^{m+n} \rightarrow \mathbb{R}$  is linear in  $(\mathbf{x}, \mathbf{y})$  and  $F, H_i, G_j, F_{i,j}$  are constant symmetric matrices of the same size. Note that multiple BMI constraints can be compressed into a single constraint using block-diagonal matrices. In next section, we will invoke the DC algorithm as a black-box and readers can skip Sect. 4.1 and Sect. 4.2.

#### 4.1 Difference-of-Convex Decomposition

Using matrix notations, we express  $\mathcal{B}(\mathbf{x}, \mathbf{y})$  in a more condensed form:

$$\mathcal{B}(\mathbf{x}, \mathbf{y}) = \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix}^\top M \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix} + (\Omega_1 \ \Omega_2) \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix} + F \quad (20)$$

where

$$\begin{aligned} M &= \begin{pmatrix} \mathbf{0} & \Gamma \\ \Gamma & \mathbf{0} \end{pmatrix} \text{ with } \Gamma = \frac{1}{2} \begin{pmatrix} F_{1,1} & \dots & F_{1,n} \\ \vdots & \ddots & \vdots \\ F_{m,1} & \dots & F_{m,n} \end{pmatrix}, \\ \Omega_1 &= (H_1 \dots H_m), \quad \Omega_2 = (G_1 \dots G_n). \end{aligned} \quad (21)$$

Here,  $\otimes$  denotes the tensor product, while  $I$  and  $\mathbf{0}$  represent identity and zero matrices of appropriate sizes, respectively.

A matrix-valued function  $\mathcal{B}$  over  $\mathbb{R}^n$  is *PSD-convex* if for all  $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^n$  and  $c \in (0, 1)$ ,

$$\mathcal{B}(c\mathbf{x}_1 + (1-c)\mathbf{x}_2) \preceq c\mathcal{B}(\mathbf{x}_1) + (1-c)\mathcal{B}(\mathbf{x}_2). \quad (22)$$

If the matrix  $M$  in (21) is PSD, then  $\mathcal{B}(\mathbf{x}, \mathbf{y})$  is PSD-convex, eliminating the need for DC decomposition. However, as shown in [51, Thm. 1],  $M$  is not guaranteed to be PSD in general. To circumvent this issue, we aim to decompose  $M$  as  $M = M_1 - M_2$ , where both  $M_1$  and  $M_2$  are PSD. This allows us to express  $\mathcal{B}(\mathbf{x}, \mathbf{y})$  as

$$\mathcal{B}(\mathbf{x}, \mathbf{y}) = \mathcal{B}^+(\mathbf{x}, \mathbf{y}) - \mathcal{B}^-(\mathbf{x}, \mathbf{y}), \quad (23)$$

where

$$\begin{aligned} \mathcal{B}^+(\mathbf{x}, \mathbf{y}) &\triangleq \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix}^\top M_1 \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix} + (\Omega_1 \ \Omega_2) \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix} + F, \\ \mathcal{B}^-(\mathbf{x}, \mathbf{y}) &\triangleq \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix}^\top M_2 \begin{pmatrix} \mathbf{x} \otimes I \\ \mathbf{y} \otimes I \end{pmatrix} \end{aligned} \quad (24)$$

are both PSD-convex.

One way to achieve this decomposition involves *eigendecomposition*. First, we factor  $M$  into  $M = V^\top D V$ , where  $V$  is an orthogonal matrix of eigenvectors and  $D$  is a diagonal matrix of eigenvalues. Let  $D^+$  denote the diagonal matrix obtained by setting all negative elements of  $D$  to zero and  $D^- \triangleq D^+ - D$ . We have

$$M = \underbrace{V^\top D^+ V}_{M_1} - \underbrace{V^\top D^- V}_{M_2}. \quad (25)$$

Both  $M_1$  and  $M_2$  are PSD, as  $D^+$  and  $D^-$  are diagonal matrices with non-negative entries. It is important to note that this decomposition is not unique, as discussed in [49, Sec. 5].

## 4.2 Successive Convexification

Subsequent to DC decomposition, a *successive convexification* procedure is employed to iteratively compute a sequence of feasible solutions  $\mathbf{z}_0 \triangleq (\mathbf{x}_0, \mathbf{y}_0)$ ,  $\mathbf{z}_1 \triangleq (\mathbf{x}_1, \mathbf{y}_1)$ ,  $\dots$  converging to a local optimal point of the original BMI problem, starting from an initial feasible point  $\mathbf{z}_0$ .

At the  $(k+1)$ -th iteration for any  $k \in \mathbb{N}$ , given the current feasible point  $\mathbf{z}_k$ , the next point  $\mathbf{z}_{k+1}$  is obtained by solving the following program:

$$\begin{aligned} \max_{\mathbf{z}=(\mathbf{x},\mathbf{y})} \quad & \mu(\mathbf{z}) + \frac{1}{2}\delta\|\mathbf{z} - \mathbf{z}_k\|^2 \\ \text{s.t.} \quad & \mathcal{B}^+(\mathbf{z}) - \left( \mathcal{B}^-(\mathbf{z}_k) + \langle \nabla \mathcal{B}^-(\mathbf{z}_k), \mathbf{z} - \mathbf{z}_k \rangle \right) \preceq 0, \end{aligned} \quad (26)$$

where the function  $\mathcal{B}^-$  is linearized at the current iterate  $\mathbf{z}_k$ . The regularization term  $\frac{1}{2}\delta\|\mathbf{z} - \mathbf{z}_k\|^2$  with  $\delta < 0$  ensures strict monotonic improvement in the objective function at each iteration until convergence. Although the constraint in (26) retains quadratic terms in  $\mathcal{B}^+(\mathbf{z})$ , its PSD-convexity and the linearization of  $\mathcal{B}^-$  allow it to be transformed into an equivalent LMI constraint using Schur complement, enabling the use of off-the-shelf SDP solvers [49, Thm. 8].

A notable characteristic of the DC algorithm for BMI is its guaranteed termination in terms of Karush-Kuhn-Tucker (KKT) conditions. These conditions are necessary and, under certain regularity assumptions, sufficient for a point to be a local optimum of a non-convex optimization problem [9, Sec. 5.5.3].

**Theorem 4 (Termination and convergence [49, Thm. 10]).** *If (19) has finitely many KKT points, then (1) for  $\epsilon > 0$ , algorithm Algorithm 1 terminates; (2) for  $\epsilon = 0$ , algorithm Algorithm 1 visits an infinite sequence of solutions  $\{\mathbf{z}_i\}_{i \in \mathbb{N}}$  converging to a KKT point.*

## 4.3 Limitations

The overall DC algorithm is presented in Algorithm 1, stated for the barrier certificate synthesis problem. Its limitations mainly involve two aspects.

**Algorithm 1:** DC, DC Algorithm for Barrier Certificate Synthesis

---

**Input** : A system (1) with sets  $\mathcal{I}, \mathcal{U}, \mathcal{D}$ , a barrier certificate template  $B(\mathbf{x}; \mathbf{b})$ , and a multiplier template  $\lambda(\mathbf{x}; \mathbf{a})$  with an initial guess  $\mathbf{a}_0, \mathbf{s}_0$ .

**Params.:** The degree bound  $D_b$  for formulating (18), which is set according to the degree of  $B(\mathbf{x}; \mathbf{b})$ , the maximum of iterations *maxRound* and the threshold  $\epsilon$  between two iterations.

**Output** : A solution  $(\mathbf{a}^*, \mathbf{b}^*)$ , or otherwise  $\perp$  indicating a failure.

```

1 deg  $B(\mathbf{x}) \leftarrow 1$ ;
2 repeat
3    $\triangleright$  Initialize.
4   Formulate the BMI optimization problem as in (18);
5    $\mathbf{b}_0 \leftarrow$  solve (18) as LMI by fixing  $\mathbf{a} = \mathbf{a}_0, \mathbf{s} = \mathbf{s}_0$ ;
6   if  $\mathbf{b}_0 = \perp$  then return  $\perp$ ;
7    $\triangleright$  DC decomposition.
8    $(\mu, \mathcal{B}) \leftarrow$  translate (18) in to the form of (19);
9    $(\mathcal{B}^+, \mathcal{B}^-) \leftarrow$  DC decomposition of  $\mathcal{B}$ ;
10   $\triangleright$  Successive Convexification.
11   $\mathbf{z}_0 \leftarrow (\mathbf{a}_0, \mathbf{b}_0, \mathbf{s}_0)$ ;
12  repeat
13     $\mathbf{z}_{k+1} \leftarrow$  solve the convex sub-problem (26) from  $(\mathcal{B}^+, \mathcal{B}^-)$ ;
14     $k \leftarrow k + 1$ ;
15  until  $\|\mathbf{z}_k - \mathbf{z}_{k-1}\| < \epsilon$  or  $k > \text{maxRound}$ ;
16  if correctness of  $B(\mathbf{x}; \mathbf{b}_k)$  can be verified then return  $\mathbf{b}_k$ ;
17  deg  $B(\mathbf{x}) \leftarrow \deg(B) + 1$ ;
18 until deg  $B(\mathbf{x}) > D_b$ ;
19 return  $\perp$ 

```

---

First, Algorithm 1 does not provide a method for selecting the template of the multiplier polynomial  $\lambda(\mathbf{x})$ . This necessitates either leveraging prior knowledge about the system to select templates or performing a search procedure from simple templates to complex templates, for example, in terms of the degree of the polynomials. This is similar to the procedure of searching for the template of  $B(\mathbf{x})$ .

Second, the output of Algorithm 1 is only a local optimum or a point close to a local optimum, potentially leading to suboptimal solutions to the original program (18). This is a fundamental limitation of DC algorithms and many other non-convex optimization algorithms due to the inherent computational complexity of non-convex optimization problems. To mitigate this limitation, [49] proposes a branch-and-bound framework to search for the global optimum. This approach partitions the search space and aims to find a near-optimal solution through sufficiently fine-grained partitioning. However, this method is practical only when the dimension of the template variables  $\mathbf{b}$  is relatively small.

## 5 Combining Interpolation and DC Programming

In this section, we propose a heuristic but practical solution for the aforementioned two limitations. The basic idea is to utilize our observation that the multiplier polynomial  $\lambda(\mathbf{x})$  should be an interpolant to simplify the input of the DC algorithm.

However, merely synthesizing an interpolant via solving (11) is not enough, as the position of  $\lambda(\mathbf{x})$  will also influence the expressiveness of barrier certificate conditions. To reduce conservativeness, we want  $\lambda(\mathbf{x})$  to approximate the target  $B(\mathbf{x})$  as close as possible. For example, the extreme case will be that  $\lambda(\mathbf{x}) = B(\mathbf{x})$ , so that  $\lambda(\mathbf{x})B(\mathbf{x})$  becomes  $(B(\mathbf{x}))^2$ , yielding the ideal case shown in Fig. 1c. So, our goal is to construct a procedure to search for a “good” interpolation  $\lambda(\mathbf{x})$  that is fed into Algorithm 1 as an initial point, and the challenge lies in how to control or adjust the position of  $\lambda(\mathbf{x})$  while solving (11).

Let us denote

$$p_{\mathcal{I}}(\mathbf{x}) : \mathcal{D} \rightarrow \mathbb{R}, \mathbf{x} \mapsto \max\{-g_0^{\mathcal{I}}(\mathbf{x}), -g_1^{\mathcal{I}}(\mathbf{x}), \dots, -g_{m_i}^{\mathcal{I}}(\mathbf{x})\}. \quad (27)$$

As a consequence, the initial set can be expressed as the zero sub-level set of  $p_{\mathcal{I}}(\mathbf{x})$ , i.e.,  $\mathcal{I} = \{\mathbf{x} \in \mathcal{D} \mid p_{\mathcal{I}}(\mathbf{x}) \leq 0\}$ , and we have

$$\lambda(\mathbf{x}) < p_{\mathcal{I}}(\mathbf{x}) \text{ over } \mathcal{D} \implies \lambda(\mathbf{x}) < 0 \text{ over } \mathcal{I}. \quad (28)$$

Intuitively, if we can minimize the distance between  $p_{\mathcal{I}}(\mathbf{x})$  and  $\lambda(\mathbf{x})$ , then  $\lambda(\mathbf{x}) \leq 0$  will give a close over-approximation of  $\mathcal{I}$ . So, we equipped (11) with an objective function

$$\min \int_{\mathcal{D}} |p_{\mathcal{I}}(\mathbf{x}) - \lambda(\mathbf{x})| \, d\mathbf{x}, \quad (29)$$

which is equivalent to

$$\max \int_{\mathcal{D}} \lambda(\mathbf{x}) \, d\mathbf{x} \quad (30)$$

as  $p_{\mathcal{I}}(\mathbf{x})$  is fixed. Note that  $p_{\mathcal{I}}(\mathbf{x})$  by its construction is still a continuous function over  $\mathcal{D}$ . Later, we will show that the objective function can be simplified into a linear expression when the domain  $\mathcal{D}$  is of some simple shapes.

One remaining question is how close  $\lambda(\mathbf{x})$  and  $p_{\mathcal{I}}(\mathbf{x})$  can be. In fact, they can be arbitrarily close as long as  $\lambda(\mathbf{x})$  is a polynomial of degree large enough. This follows from Theorem 5 below.

**Theorem 5.** *Let  $\mathcal{D} \subset \mathbb{R}^n$  be a compact set and  $p(\mathbf{x}) : \mathcal{D} \rightarrow \mathbb{R}$  be a bounded and continuous function. Then, there exists a sequence of polynomials  $\{\lambda_k(\mathbf{x})\}_{k \in \mathbb{N}} \subset \mathbb{R}[\mathbf{x}]$  such that*

$$\lim_{k \rightarrow \infty} \int_{\mathcal{D}} |p(\mathbf{x}) - \lambda_k(\mathbf{x})| \, d\mathbf{x} = 0.$$

*Proof.* This is a direct consequence of [30, Thm. 1], by noting that  $p_{\mathcal{I}}(\mathbf{x})$  by its construction is a continuous function over  $\mathcal{D}$ , and a continuous function is also *upper* (resp. *lower*) *semi-continuous*, i.e., for any  $\mathbf{x}_0 \in \mathcal{D}$   $\limsup_{\mathbf{x} \rightarrow \mathbf{x}_0} p(\mathbf{x}) \leq p(\mathbf{x}_0)$  (resp.  $\liminf_{\mathbf{x} \rightarrow \mathbf{x}_0} p(\mathbf{x}) \geq p(\mathbf{x}_0)$ ).  $\square$

Symmetrically, to obtain an interpolant  $\lambda(\mathbf{x})$  close to the unsafe set  $\mathcal{U}$ , we solve (11) with the objective

$$\min \int_{\mathcal{D}} \lambda(\mathbf{x}) \, d\mathbf{x}. \quad (31)$$

In practice, since the multiplier polynomial  $\lambda(\mathbf{x})$  is parameterized as  $\lambda(\mathbf{x}; \mathbf{a}) = \sum_{\alpha} a_{\alpha} \mathbf{x}^{\alpha}$ , where  $\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{N}^n$  denotes the exponent of the monomial  $\mathbf{x}^{\alpha}$ , the integral in (30) and (31) can be computed as

$$\begin{aligned} \int_{\mathcal{D}} \lambda(\mathbf{x}; \mathbf{a}) \, d\mathbf{x} &= \int_{\mathcal{D}} \sum_{\alpha} a_{\alpha} \mathbf{x}^{\alpha} \, d\mathbf{x} \\ &= \sum_{\alpha} a_{\alpha} \left( \int_{\mathcal{D}} \mathbf{x}^{\alpha} \, d\mathbf{x} \right), \end{aligned} \quad (32)$$

which is a linear expression in parameter variables  $\mathbf{a}$ . For simplicity, we will assume that the set  $\mathcal{D}$  is of some simple shapes, e.g., boxes or ellipsoids, so that  $\int_{\mathcal{D}} \mathbf{x}^{\alpha} \, d\mathbf{x}$  can be efficiently computed. For example, consider  $\mathcal{D} = [-1, 1]^n$ , then

$$\int_{\mathcal{D}} \mathbf{x}^{\alpha} \, d\mathbf{x} = \begin{cases} 0 & \text{if some } \alpha_i \text{ is odd.} \\ 2^n \prod_{i=1}^n (\alpha_i + 1)^{-1} & \text{o.w.} \end{cases} \quad (33)$$

Now, we are ready to present our new algorithm **ltp-DC** as in Algorithm 2. In line 4 to 5, we first solve (11) with the objective (30) (resp. (31)) to obtain a polynomial interpolant  $\lambda_1^*(\mathbf{x})$  (resp.  $\lambda_2^*(\mathbf{x})$ ) that is closest to the initial set  $\mathcal{I}$  (resp. the unsafe set  $\mathcal{U}$ ). The degree of  $\lambda_1^*(\mathbf{x})$  and  $\lambda_2^*(\mathbf{x})$  is searched from 1 to the degree of  $B(\mathbf{x}; \mathbf{b})$ . If no interpolant is found at the degree of template  $B(\mathbf{x}; \mathbf{b})$ , this directly implies that there is no solution for the template  $B(\mathbf{x}; \mathbf{b})$ , as a barrier certificate is also an interpolant between  $\mathcal{I}$  and  $\mathcal{U}$ . Note that all interpolants between  $\mathcal{I}$  and  $\mathcal{U}$  forms a convex set, so we can set  $\lambda(\mathbf{x})$  to be of the form

$$\lambda(\mathbf{x}; \mathbf{a}_0) = c\lambda_1^*(\mathbf{x}) + (1 - c)\lambda_2^*(\mathbf{x}) \quad (34)$$

where  $c \in [0, 1]$  determines the position of the interpolant (line 10), taking its values from a set of predefined values in  $[0, 1]$ . In our experiments, we set  $V_c = \{0, 0.25, 0.5, 0.75, 1\}$ . From line 9 to 13, we construct  $\lambda(\mathbf{x}; \mathbf{a})$  as above and feed it to Algorithm 1 with an initial guess  $\mathbf{a} = \mathbf{a}_0$ .

## 6 Case Study

In this section, we present a case study to show the effectiveness of Algorithm 2. We implemented our Algorithm 2 based on the original DC algorithm, using Wolfram Mathematica (14.1.0.0) [25] to formulate the BMI constraints and SDP solver Mosek [6] to solve the LMI in each iteration. For comparison, we have also implemented the base-line method (denoted the Exp algorithm) based on exponential-type barrier certificates from [28], the alternating descent

**Algorithm 2:** ltp-DC

---

**Input** : A system (1) with sets  $\mathcal{I}, \mathcal{U}, \mathcal{D}$ ; a BC template  $B(\mathbf{x}; \mathbf{b})$ .  
**Params.**: A set of optional values  $V_c$  for  $c$ .  
**Output** : A valid parameter  $\mathbf{b}^*$ , or otherwise  $\perp$  indicating a failure.

---

```

1 ▷ search for an interpolant  $\lambda(\mathbf{x})$  as the multiplier
2  $d \leftarrow 1$ ;
3 repeat
4    $\lambda_1^*(\mathbf{x}) \in \mathbb{R}^d \leftarrow$  solve (11) with the objective (30);
5    $\lambda_2^*(\mathbf{x}) \in \mathbb{R}^d \leftarrow$  solve (11) with the objective (31);
6    $d \leftarrow d + 1$ 
7 until ( $\lambda_1^*(\mathbf{x}) \neq \perp$  and  $\lambda_2^*(\mathbf{x}) \neq \perp$ ) or  $d > \deg B(\mathbf{x})$ ;
8 if  $d > \deg B(\mathbf{x})$  then return  $\perp$ ;
9 for  $c$  in  $V_c$  do
10   $\lambda(\mathbf{x}; \mathbf{a}_0) \leftarrow c\lambda_1^* + (1 - c)\lambda_2^*$ ;
11   $\mathbf{b}^* \leftarrow$  invoke DC Algorithm 1 with  $\lambda(\mathbf{x}; \mathbf{a})$  and an initial guess  $\mathbf{a} = \mathbf{a}_0$ ;
12  if correctness of  $B(\mathbf{x}; \mathbf{b}^*)$  can be verified then return  $\mathbf{b}^*$ ;
13 end
14 return  $\perp$ 

```

---

(AD) algorithm from [56], together with the DC Algorithm 1 without using interpolation from [49]. The computation is performed on a 4.05GHz on Apple M3 chips processor with 8 GB RAM running MacOS Sonoma 14.6.1.

Consider the following dynamical system,

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix} = \begin{pmatrix} x_2 + 2x_1x_2 \\ -x_1 - x_2^2 + 2x_1^2 \end{pmatrix} \quad (35)$$

with  $\mathcal{D} = \{(x_1, x_2) \in \mathbb{R}^2 \mid -3 \leq x_1 \leq 3, -3 \leq x_2 \leq 3\}$ ,  $\mathcal{I} = \{(x_1, x_2) \in \mathbb{R}^2 \mid 0.25 \leq x_1 \leq 2.25, 0.25 \leq x_2 \leq 2.25\}$ , and  $\mathcal{U} = \{(x_1, x_2) \in \mathbb{R}^2 \mid -2.5 \leq x_1 \leq -0.5, -2.5 \leq x_2 \leq -0.5\}$ .

For all algorithms, we search for a barrier certificate from degree 1 to degree  $D_b = 5$ , where in practice  $D_b \in \mathbb{N}$  can be a user-defined upper bound. When formulating the BMI/LMI constraints, we set  $\epsilon_0 = 10^{-5}$  and require all the unknown coefficients  $\{\mathbf{a}, \mathbf{b}, \mathbf{s}\}$  to range from  $-1$  to  $1$ . The degree of unknown SOS polynomials are set according to the largest degree of polynomials in the constraint. For example, the degree of SOS polynomial  $\sigma_i^T(x)$  is set to be the minimum even value not less than  $\deg B(\mathbf{x}) - \deg g_i^T + 2$ , where 2 is a user-defined degree tolerance. The degree of  $\lambda(x)$  is set to be 1, i.e.  $\lambda(\mathbf{x}; \mathbf{a})$  is a linear function. For the DC algorithm and the AD algorithm, we sample 5 different initial values for  $\mathbf{a}$  at each degree. For the Exp algorithm,  $\lambda \in \mathbb{N}$  is a constant and we try different values in  $\{-1, -0.5, 0, 0.5, 1\}$ . The timeout is set to be 5 minutes, and for BMI-based methods the maximum number of iterations is set to be 20.

Using Algorithm 2, we successfully synthesized an barrier certificate of degree 4 in 26.4 seconds. The original DC-Algorithm failed to find a barrier certificate of degree 4 but succeeded at degree 5, taking 291.5 seconds. In comparison, both

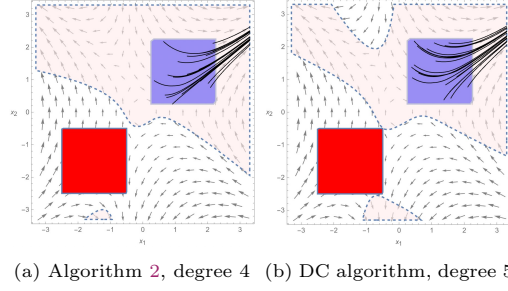


Fig. 2: Portraits of Results

the Exp algorithm and the AM algorithm failed to find a barrier certificate within the search range, spending a time of 12.2 seconds and 48.8 seconds, respectively. The results of Algorithm 2 and original DC-Algorithm are displayed in Fig. 2. The blue and red regions represent the initial and unsafe regions, respectively; the gray arrows represent the flow directions of the dynamical system, the black curves represent the trajectory start from the initial region, and the pink region surrounded by dotted lines is the region of  $B(x) \leq 0$ .

In summary, compared with the exponential-type barrier certificate conditions, our Algorithm 2 relaxes the Lie derivative condition by replacing the fixed constant  $\lambda \in \mathbb{R}$  with an interpolant polynomial  $\lambda(\mathbf{x})$ , thereby gaining more expressiveness. In contrast to the original DC algorithm, our Algorithm 2 is more efficient as it utilizes an initial point based on the initial and unsafe regions, rather than relying on random initialization. Moreover, our Algorithm 2 preserves the convergence guarantee to local optimums, which is not the case for the AD algorithm.

## 7 Conclusion

This paper revisits the primary approaches to encoding barrier certificate conditions and identifies a special case where the multiplier polynomial  $\lambda(\mathbf{x})$  can be interpreted as an interpolant between  $\mathcal{I}$  and  $\mathcal{U}$ . In this case, the barrier certificate conditions can be made less conservative, while  $\lambda(\mathbf{x})$  can still be obtained through SOS techniques. By introducing an integral as an optimization objective, we can control the position of the interpolant through a single variable  $c \in [0, 1]$ . The obtained interpolant is then provided as an initial value to the DC algorithm, thereby alleviating the challenges associated with both the template setting problem for  $\lambda(\mathbf{x})$  and the initial value problem for the DC algorithm.

In future work, we plan to extend these types of barrier certificate conditions to control systems and delay systems [16]. For example, when synthesizing control barrier functions based on SOS techniques [2], we believe that our extensions can yield less conservative results than the exponential-type conditions (which is the default choice). Additionally, we are committed to conducting a detailed and comprehensive comparison with existing optimization-based and neural network-based approaches.

## References

1. Althoff, M., Frehse, G., Girard, A.: Set propagation techniques for reachability analysis. *Annual Review of Control, Robotics, and Autonomous Systems* **4**, 369–395 (2021)
2. Ames, A.D., Xu, X., Grizzle, J.W., Tabuada, P.: Control barrier function based quadratic programs for safety critical systems. *IEEE Trans. Autom. Control.* **62**(8), 3861–3876 (2017)
3. Anai, H., Weispfenning, V.: Reach set computations using real quantifier elimination. In: HSCC. LNCS, vol. 2034, pp. 63–76. Springer (2001)
4. Anand, M., Murali, V., Trivedi, A., Zamani, M.: Safety verification of dynamical systems via k-inductive barrier certificates. In: CDC. pp. 1314–1320. IEEE (2021)
5. Anand, M., Murali, V., Trivedi, A., Zamani, M.: k-inductive barrier certificates for stochastic systems. In: HSCC. pp. 12:1–12:11. ACM (2022)
6. ApS, M.: The MOSEK optimization toolbox for MATLAB manual. Version 9.0. (2019), <http://docs.mosek.com/9.0/toolbox/index.html>
7. Bak, S.: *t*-barrier certificates: A continuous analogy to *k*-induction. In: ADHS. IFAC-PapersOnLine, vol. 51, pp. 145–150. Elsevier (2018)
8. Barbosa, H., Barrett, C.W., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: TACAS (1). LNCS, vol. 13243, pp. 415–442. Springer (2022)
9. Boyd, S., Boyd, S.P., Vandenberghe, L.: Convex optimization. Cambridge university press (2004)
10. Chatterjee, K., Goharshady, A.K., Goharshady, E.K., Karrabi, M., Zikelic, D.: Sound and complete witnesses for template-based verification of LTL properties on polynomial programs. In: FM (1). LNCS, vol. 14933, pp. 600–619. Springer (2024)
11. Chatterjee, K., Henzinger, T.A., Lechner, M., Zikelic, D.: A learner-verifier framework for neural network controllers and certificates of stochastic systems. In: TACAS (1). LNCS, vol. 13993, pp. 3–25. Springer (2023)
12. Chen, X., Peng, C., Lin, W., Yang, Z., Zhang, Y., Li, X.: A novel approach for solving the BMI problem in barrier certificates generation. In: CAV (1). LNCS, vol. 12224, pp. 582–603. Springer (2020)
13. Dai, L., Gan, T., Xia, B., Zhan, N.: Barrier certificates revisited. *J. Symb. Comput.* **80**, 62–86 (2017)
14. Davenport, J.H., Heintz, J.: Real quantifier elimination is doubly exponential. *Journal of Symbolic Computation* **5**(1-2), 29–35 (1988)
15. Edwards, A., Peruffo, A., Abate, A.: Fossil 2.0: Formal certificate synthesis for the verification and control of dynamical models. In: HSCC. pp. 26:1–26:10. ACM (2024)
16. Feng, S., Chen, M., Zhan, N., Fränzle, M., Xue, B.: Taming delays in dynamical systems - unbounded verification of delay differential equations. In: CAV (1). LNCS, vol. 11561, pp. 650–669. Springer (2019)
17. Fränzle, M., Chen, M., Kröger, P.: In memory of Oded Maler: Automatic reachability analysis of hybrid-state automata. *ACM SIGLOG News* **6**(1), 19–39 (2019)
18. Gan, T., Chen, M., Li, Y., Xia, B., Zhan, N.: Reachability analysis for solvable dynamical systems. *IEEE Trans. Autom. Control.* **63**(7), 2003–2018 (2018)

19. Gan, T., Xia, B., Xue, B., Zhan, N., Dai, L.: Nonlinear craig interpolant generation. In: CAV. LNCS, vol. 12224, pp. 415–438. Springer (2020)
20. Gao, S., Avigad, J., Clarke, E.M.:  $\delta$ -complete decision procedures for satisfiability over the reals. In: IJCAR. LNCS, vol. 7364, pp. 286–300. Springer (2012)
21. Gao, S., Kong, S., Clarke, E.M.: dreal: An SMT solver for nonlinear theories over the reals. In: CADE. LNCS, vol. 7898, pp. 208–214. Springer (2013)
22. Henzinger, T.A., Kopke, P.W., Puri, A., Varaiya, P.: What’s decidable about hybrid automata? In: STOC. pp. 373–382. ACM (1995)
23. Huang, C., Chen, X., Lin, W., Yang, Z., Li, X.: Probabilistic safety verification of stochastic hybrid systems using barrier certificates. *ACM Transactions on Embedded Computing Systems* **16**(5s), 186:1–186:19 (2017)
24. Huang, L., Nie, J., Yuan, Y.: Homogenization for polynomial optimization with unbounded sets. *Mathematical Programming* **200**(1), 105–145 (2023)
25. Inc., W.R.: Mathematica, Version 14.1, <https://www.wolfram.com/mathematica>, champaign, IL, 2024
26. Jagtap, P., Soudjani, S., Zamani, M.: Formal synthesis of stochastic systems via control barrier certificates. *IEEE Transactions on Automatic Control* **66**(7), 3097–3110 (2021)
27. Kapinski, J., Deshmukh, J.V., Sankaranarayanan, S., Aréchiga, N.: Simulation-guided lyapunov analysis for hybrid dynamical systems. In: HSCC. pp. 133–142. ACM (2014)
28. Kong, H., He, F., Song, X., Hung, W.N.N., Gu, M.: Exponential-condition-based barrier certificate generation for safety verification of hybrid systems. In: CAV. LNCS, vol. 8044, pp. 242–257. Springer (2013)
29. Lafferriere, G., Pappas, G.J., Yovine, S.: Symbolic reachability computation for families of linear vector fields. *Journal of Symbolic Computation* **32**(3), 231–253 (2001)
30. Lasserre, J.B.: Tractable approximations of sets defined with quantifiers. *Mathematical Programming* **151**(2), 507–527 (2015)
31. Lasserre, J.B.: Moments, positive polynomials and their applications, vol. 1. World Scientific (2009)
32. Liu, J., Zhan, N., Zhao, H.: Computing semi-algebraic invariants for polynomial dynamical systems. In: EMSOFT. pp. 97–106. ACM (2011)
33. de Moura, L.M., Bjørner, N.S.: Z3: an efficient SMT solver. In: TACAS. LNCS, vol. 4963, pp. 337–340. Springer (2008)
34. Murali, V., Trivedi, A., Zamani, M.: Closure certificates. In: HSCC. pp. 10:1–10:11. ACM (2024)
35. Peruffo, A., Ahmed, D., Abate, A.: Automated and formal synthesis of neural barrier certificates for dynamical models. In: TACAS (1). LNCS, vol. 12651, pp. 370–388. Springer (2021)
36. Platzer, A.: *Logical Analysis of Hybrid Systems*. Springer (2010)
37. Platzer, A., Clarke, E.M.: Computing differential invariants of hybrid systems as fixedpoints. In: CAV. LNCS, vol. 5123, pp. 176–189. Springer (2008)
38. Podelski, A., Rybalchenko, A.: Transition invariants. In: LICS. pp. 32–41. IEEE Computer Society (2004)
39. Prajna, S., Jadbabaie, A.: Safety verification of hybrid systems using barrier certificates. In: HSCC. LNCS, vol. 2993, pp. 477–492. Springer (2004)
40. Prajna, S., Jadbabaie, A., Pappas, G.J.: A framework for worst-case and stochastic safety verification using barrier certificates. *IEEE Transactions on Automatic Control* **52**(8), 1415–1428 (2007)

41. Putinar, M.: Positive polynomials on compact semi-algebraic sets. *Indiana University Mathematics Journal* **42**(3), 969–984 (1993)
42. Sogokon, A., Ghorbal, K., Tan, Y.K., Platzer, A.: Vector barrier certificates and comparison systems. In: FM. LNCS, vol. 10951, pp. 418–437. Springer (2018)
43. Taly, A., Tiwari, A.: Deductive verification of continuous dynamical systems. In: FSTTCS. LIPIcs, vol. 4, pp. 383–394. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2009)
44. Tao, P.D., Souad, E.B.: Algorithms for solving a class of nonconvex optimization problems. methods of subgradients. *North-holland Mathematics Studies* **129**, 249–271 (1986)
45. Tarski, A.: A decision method for elementary algebra and geometry. University of California Press, Berkeley (May 1951)
46. Thi, H.A.L., Dinh, T.P.: Open issues and recent advances in DC programming and DCA. *J. Glob. Optim.* **88**(3), 533–590 (2024)
47. Walter, W.: Ordinary differential equations, vol. 182. Springer Science & Business Media (1998)
48. Wang, Q., Chen, M., Xue, B., Zhan, N., Katoen, J.: Synthesizing invariant barrier certificates via difference-of-convex programming. In: CAV (1). LNCS, vol. 12759, pp. 443–466. Springer (2021)
49. Wang, Q., Chen, M., Xue, B., Zhan, N., Katoen, J.: Encoding inductive invariants as barrier certificates: Synthesis via difference-of-convex programming. *Information and Computation* **289**(Part), 104965 (2022)
50. Wang, Q., Li, Y., Xia, B., Zhan, N.: Generating semi-algebraic invariants for non-autonomous polynomial hybrid systems. *Journal of Systems Science and Complexity* **30**(1), 234–252 (2017)
51. Wang, Y., Rajamani, R.: Feasibility analysis of the bilinear matrix inequalities with an application to multi-objective nonlinear observer design. In: CDC. pp. 3252–3257. IEEE (2016)
52. Wongpiromsarn, T., Topcu, U., Lamperski, A.G.: Automata theory meets barrier certificates: Temporal logic verification of nonlinear systems. *IEEE Trans. Autom. Control*. **61**(11), 3344–3355 (2016)
53. Wu, H., Feng, S., Gan, T., Wang, J., Xia, B., Zhan, N.: On completeness of sdp-based barrier certificate synthesis over unbounded domains. In: FM (2). LNCS, vol. 14934, pp. 248–266. Springer (2024)
54. Xu, X., Tabuada, P., Grizzle, J.W., Ames, A.D.: Robustness of control barrier functions for safety critical control. In: ADHS. IFAC-PapersOnLine, vol. 48, pp. 54–61. Elsevier (2015)
55. Xue, B., Zhan, N., Fränzle, M., Wang, J., Liu, W.: Reach-avoid verification based on convex optimization. *IEEE Trans. Autom. Control*. **69**(1), 598–605 (2024)
56. Yang, Z., Lin, W., Wu, M.: Exact safety verification of hybrid systems based on bilinear SOS representation. *ACM Trans. Embed. Comput. Syst.* **14**(1), 16:1–16:19 (2015)
57. Zhan, N., Wang, S., Zhao, H.: Formal Verification of Simulink/Stateflow Diagrams: A Deductive Way. Springer (2017)
58. Zhang, H., Qin, Z., Gao, S., Clark, A.: Seev: Synthesis with efficient exact verification for relu neural barrier functions (2024), <https://arxiv.org/abs/2410.20326>
59. Zhao, H., Qi, N., Dehbi, L., Zeng, X., Yang, Z.: Formal synthesis of neural barrier certificates for continuous systems via counterexample guided learning. *ACM Trans. Embed. Comput. Syst.* **22**(5s), 146:1–146:21 (2023)
60. Zhao, H., Zeng, X., Chen, T., Liu, Z.: Synthesizing barrier certificates using neural networks. In: HSCC. pp. 25:1–25:11. ACM (2020)